Logistic regression và softmax

Dự đoán xác suất để phân loại

Phase 2 · Bài 2.3

Bộ lọc thư rác

“Chúc mừng! Bạn đã trúng iPhone, bấm vào đây để nhận.”

Bộ lọc không cần biết chắc. Nó cần nói: **xác suất spam là 0.97**.

Người dùng tự chọn chặn từ mức nào.

CÂU HỎI

Linear regression cho một số thực bất kỳ.

Làm sao biến nó thành xác suất, và học nó bằng hàm mất mát nào?

Sau buổi này, các em có thể

- Giải thích vì sao linear regression không dùng được cho phân loại, đọc hệ số theo odds.
- Suy ra gradient của cross-entropy, tự cài bằng NumPy, kiểm gradient, đối chiếu scikit-learn.
- Chọn ngưỡng quyết định theo cái giá của từng loại sai.
- Cài softmax không tràn số cho bài toán nhiều lớp.

Dữ liệu

Mỗi email: **số link** và **số từ quảng cáo** (“miễn phí”, “trúng thưởng”).

Nhãn 1 là spam, 0 là thư thường.

400 email: 300 để học, 100 để kiểm tra. Khoảng một phần ba là spam.

```
z_true = 1.1 * links + 0.9 * kw - 4.5
y = rng.random(n) < 1 / (1 + np.exp(-z_true))
X = np.column_stack([links, kw])     # (400, 2)
```

Tự sinh từ quy luật đã biết để đối chiếu xem mô hình học được gì.

01

Vì sao không dùng linear regression?

Đầu ra không phải xác suất

```
lin = LinearRegression().fit(X_tr, y_tr)
p_lin = lin.predict(X_tr)
```

KẾT QUẢ

```
dự đoán nhỏ nhất: -0.2  lớn nhất: 1.34
số email có 'xác suất' ngoài [0, 1]: 54
```

"Xác suất -0.2" không có nghĩa gì.

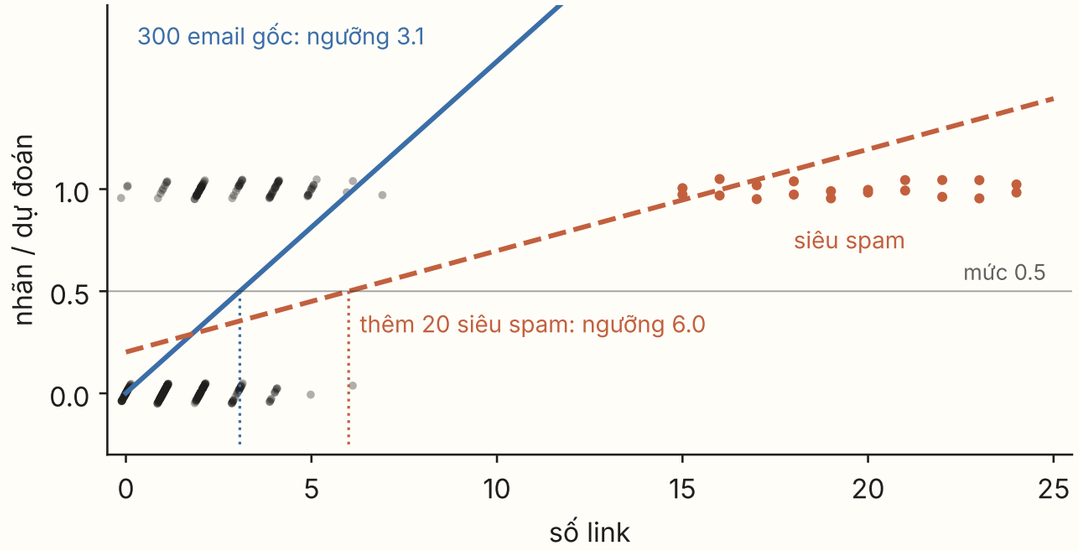
Và còn một vấn đề nghiêm trọng hơn.

Bị điểm dễ kéo lệch

Chỉ dùng số link. Thêm 20 email "siêu spam" có 15–24 link.

	Ngưỡng	Accuracy test
300 email gốc	3.07 link	0.78
Thêm 20 siêu spam	**6.00 link**	**0.68**

Hàm bình phương phạt cả chỗ **đúng mà quá chắc chắn**.



Đường thẳng thoải xuống, ngưỡng dịch từ 3 sang 6 link.

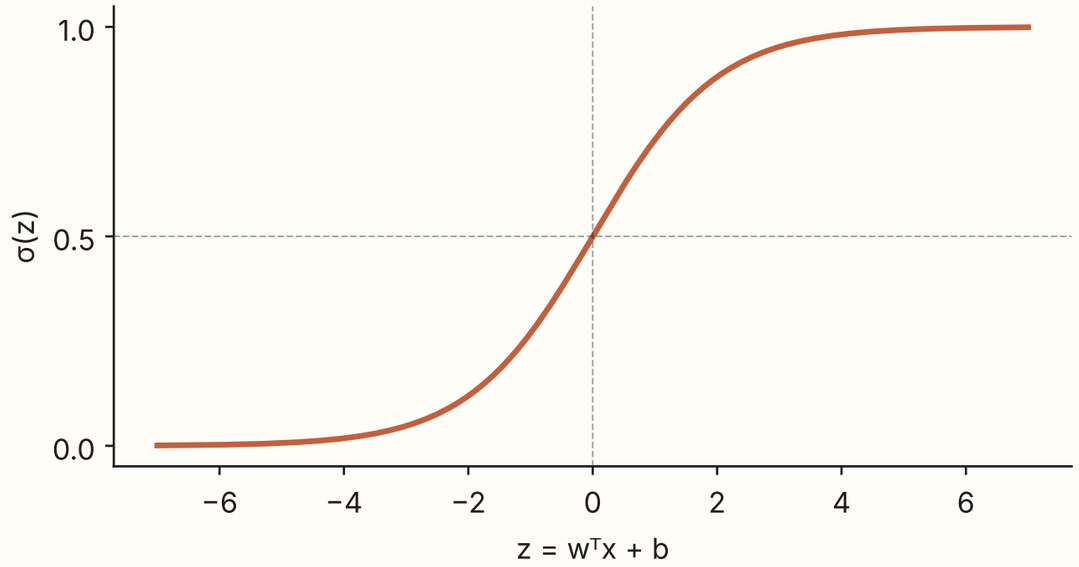
02

Mô hình

Sigmoid

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z = \mathbf{w}^\top \mathbf{x} + b$$

- Mọi số thực vào, một số trong $(0, 1)$ ra.
- $z = 0 \Rightarrow p = 0.5$; $z = 3 \Rightarrow p \approx 0.95$.



Hàm sigmoid.

Log-odds

$$z = \log \frac{p}{1-p}$$

Odds $\frac{p}{1-p}$: “mấy ăn một”. $p = 0.75$ thì odds bằng 3.

Tuyến tính cho **log-odds**, không phải cho xác suất.

Tăng x_j một đơn vị: odds nhân e^{w_j}.

HỎI NHANH

Mô hình cho một email $p = 0.8$. Odds spam bằng bao nhiêu?

A 0.8

B 0.2

C 4

D 1.25

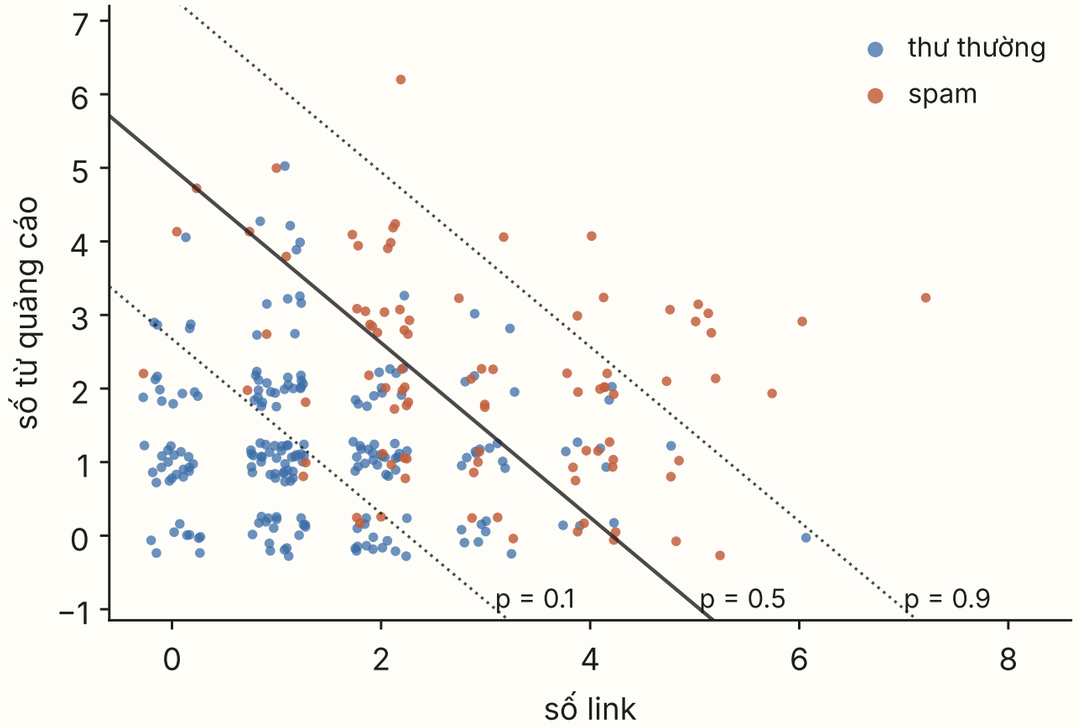
Ranh giới quyết định

$p \geq 0.5 \iff z \geq 0$, nên ranh giới là

$$\mathbf{w}^\top \mathbf{x} + b = 0.$$

Hai đặc trưng: một **đường thẳng**. Sigmoid không làm ranh giới cong.

Muốn cong: thêm $x_1^2, x_2^2, x_1 x_2$ như bài 2.1.



Ba đường song song: p = 0.1, 0.5, 0.9.

Thí nghiệm siêu spam, lần nữa

```
m = LogisticRegression(C=np.inf).fit(L, yy)
t = -m.intercept_[0] / m.coef_[0, 0]
```

	Ngưỡng	Accuracy test
300 email gốc	3.01 link	0.78
Thêm 20 siêu spam	3.01 link	0.78

Ngưỡng không nhúc nhích.

Email 24 link đã có $p \approx 1$: đúng và tự tin đúng thì gần như không bị phạt.

Lý do nằm ở hàm mất mát.

03

Cross-entropy

Từ maximum likelihood

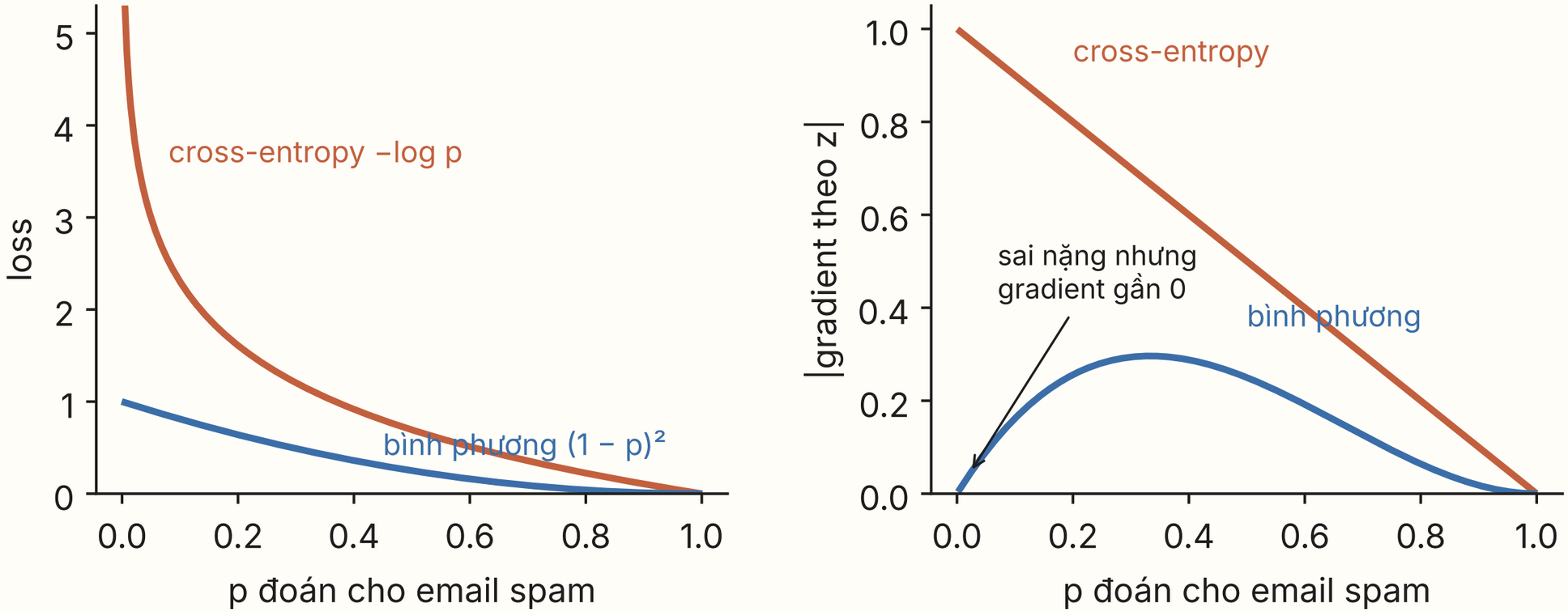
Xác suất mô hình gán cho nhãn đã xảy ra: $p_i^{y_i}(1-p_i)^{1-y_i}$

Nhân lại, lấy log, đổi dấu, chia n:

$$J = -\frac{1}{n}\sum_i \Big[y_i \log p_i + (1-y_i)\log(1-p_i)\Big]$$

Spam thật: đoán 0.99 mất 0.01 · đoán 0.5 mất 0.69 · đoán 0.01 mất 4.6

Vì sao không dùng hàm bình phương



Gần p = 0: gradient của hàm bình phương tắt dần, cross-entropy vẫn đẩy mạnh.

Spam thật, đoán $p = 0.01$: gradient theo z là -0.0196 (bình phương) so với -0.99 (cross-entropy).

04

Gradient và tự cài

Đạo hàm

$\sigma'(z) = p(1-p)$. Theo quy tắc chuỗi:

$$\frac{\partial \ell}{\partial z} = \left(-\frac{y}{p} + \frac{1-y}{1-p}\right) p(1-p) = p - y$$

$$\nabla_{\mathbf{w}} J = \frac{1}{n} X^\top (\mathbf{p} - \mathbf{y})$$

Giống linear regression: chỉ thay $\hat{\mathbf{y}}$ bằng $\mathbf{p}$.

Đại lượng	Shape
`X`	$(300, 2)$
`z = X @ w + b`	$(300,)$
`p = sigmoid(z)`	$(300,)$
`X.T @ (p - y) / n`	$(2,)$

Cài đặt và kiểm gradient

```
def loss(theta, X, y, eps=1e-12):
    p = sigmoid(X @ theta[:-1] + theta[-1])
    p = np.clip(p, eps, 1 - eps)
    return -np.mean(y * np.log(p)
                    + (1 - y) * np.log(1 - p))

def grad(theta, X, y):
    r = sigmoid(X @ theta[:-1] + theta[-1]) - y
    return np.append(X.T @ r / len(y), r.mean())
```

So với sai phân trung tâm:

KẾT QUẢ

```
sai số tương đối: 5.8e-12
```

Gradient descent, lr $= 0.5$, 5000 vòng:

KẾT QUẢ

```
w = [1.124 0.947]   b = -4.73
loss ban đầu: 0.6931   sau 5000 vòng: 0.3995
```

Đối chiếu scikit-learn

KẾT QUẢ

```
scikit-learn: w = [1.124 0.947]   b = [-4.73]
trùng gradient descent: True
```

- Quy luật thật: $(1.1,\ 0.9,\ -4.5)$.
- scikit-learn dùng L-BFGS: vài chục bước thay vì vài nghìn.
- Không có nghiệm đóng.

THAM SỐ C

Mặc định `C=1.0`: có phạt L2, `C` tỉ lệ nghịch với λ.

`C=np.inf` tắt phạt để so với code tự viết.

Dữ liệu tách được hoàn toàn mà không phạt: trọng số tăng mãi.

HỎI NHANH

Khởi tạo mọi tham số bằng 0 trên bài hai lớp. Loss ban đầu bằng bao nhiêu?

A 0

B 0.5

C $\log 2 \approx 0.693$

D Không xác định được

05

Đọc kết quả

Đọc hệ số qua odds

Thêm 1 link: odds $\times 3.08$. Thêm 1 từ quảng cáo: odds $\times 2.58$.

(số link, số từ)	P(spam)
(1, 1)	0.065
(2, 1)	0.177
(7, 1)	0.983
(8, 1)	0.995

- Odds nhân đều $\times 3$ mỗi link.
- Xác suất thì không: $1 \to 2$ link tăng 0.11 ; $7 \to 8$ link chỉ tăng 0.01.
- Odds nhân đều, xác suất bão hòa.

Ngưỡng quyết định

100 email test, 33 spam:

Ngưỡng	Chặn đúng spam	Chặn nhầm thư thật	Lọt spam
0.3	30	21	3
0.5	25	7	8
0.9	4	0	29

Accuracy 0.85; luôn đoán "không spam": 0.67.

- `predict` = `predict_proba` cắt ở 0.5.
- Ngưỡng là quyết định nghiệp vụ, theo cái giá của từng loại sai.
- So accuracy với mốc **đoán lớp đông nhất**.
- Log loss: mốc 0.635, mô hình 0.371.

HỎI NHANH

Chặn nhầm thư thật đắt hơn nhiều so với để lọt spam. Chỉnh ngưỡng thế nào?

A Hạ xuống 0.3

B Giữ 0.5

C Nâng lên, chẳng hạn 0.9

D Huấn luyện thêm nhiều vòng

06

Nhiều lớp: softmax

Softmax regression

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad z_k = \mathbf{w}_k^\top \mathbf{x} + b_k$$

- Loss: $-\log p_y$. Gradient theo logits: $\mathbf{p} - \mathbf{e}_y$.
- $K = 2$: $p_1 = \sigma(z_1 - z_2)$, chính là logistic regression.

Đại lượng	Shape
`X`	$(1300, 64)$
`W, b`	$(64, 10)$, $(10,)$
`P = softmax(X @ W + b)`	$(1300, 10)$
`X.T @ (P - Y) / n`	$(64, 10)$

Trừ max trước khi lấy mũ

e^{1000} tràn số: `inf / inf = nan`.

Cộng cùng hằng số vào mọi z_k không đổi softmax, nên trừ $\max_k z_k$.

```
def softmax(Z):
    Z = Z - Z.max(axis=-1, keepdims=True)
    E = np.exp(Z)
    return E / E.sum(axis=-1, keepdims=True)
```

KẾT QUẢ

```
[0.665 0.245 0.09 ]
[nan nan nan] [0.665 0.245 0.09 ]
```

Tính $\log p_k$ thì dùng log-sum-exp:

$$\log p_k = z_k - m - \log \sum_j e^{z_j - m}$$

Chữ số viết tay

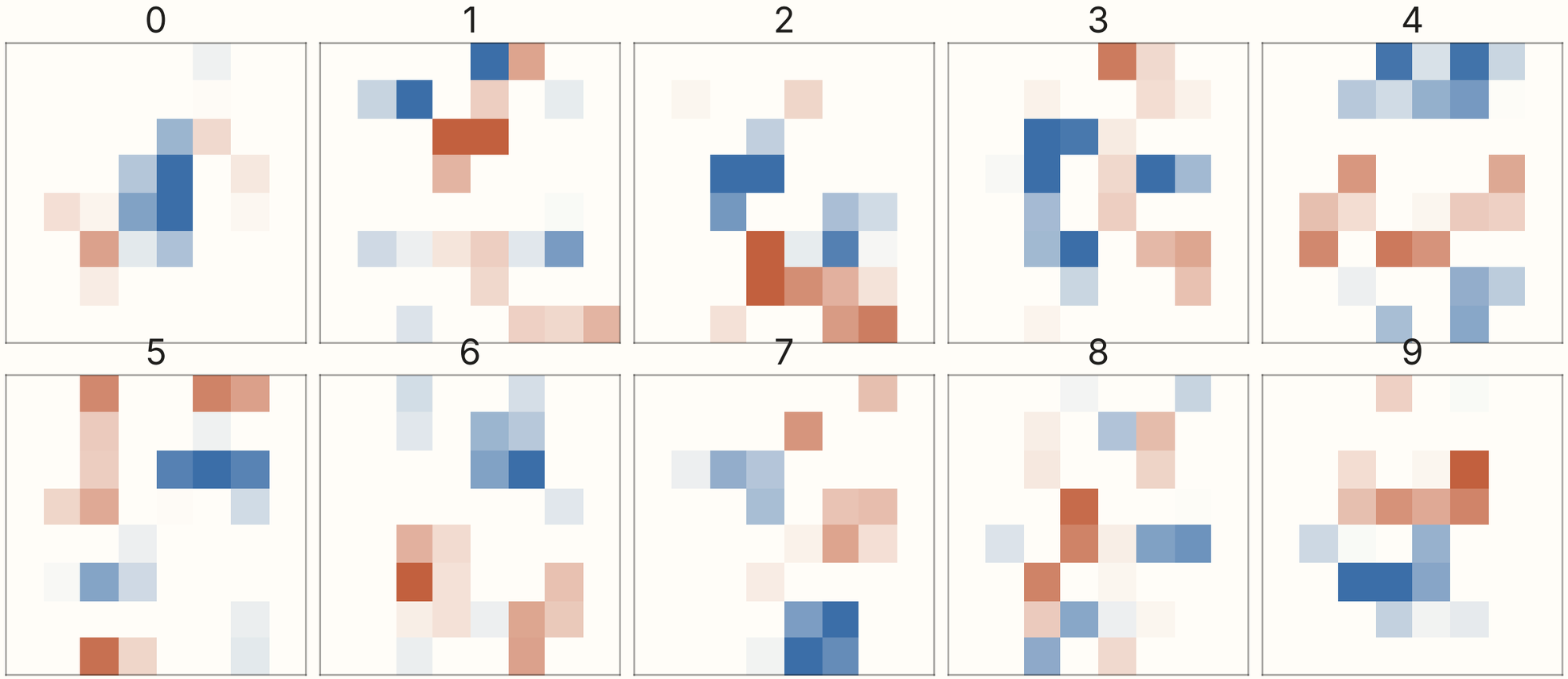
1797 ảnh 8×8, 1300 để học, 497 để kiểm tra.

Gradient descent 5000 vòng, phạt L2 khớp `LogisticRegression(C=1)`.

	Accuracy test
Tự cài	0.978
scikit-learn	0.978

Nhãn trùng trên mọi ảnh test; xác suất lệch tối đa 0.032 vì gradient descent chưa hội tụ hẳn.

Mô hình nhìn gì



Trọng số từng lớp, xếp thành ảnh 8×8: ô cam có mực thì tăng điểm, ô xanh có mực thì trừ điểm. Lớp 0 bị trừ điểm khi có nét ở giữa ảnh.

HỎI NHANH

Cộng 100 vào mọi logit. Xác suất softmax thay đổi thế nào?

A Lớp lớn nhất tiến về 1

B Mọi xác suất tăng

C Không thay đổi

D Thành nan

Tính tay trên lớp

$x = 2$, $y = 1$, $w = 1$, $b = -1$. Tính p, loss, gradient theo w.

Softmax của $\mathbf{z} = (2, 1, 0)$?

$z = 1$, $p = 0.731$, loss $= 0.313$, gradient $= (0.731 - 1) \cdot 2 = -0.538$.

Trừ max: $(0, -1, -2)$ → mũ: $(1,\ 0.368,\ 0.135)$ → chia tổng 1.503:

$$\mathbf{p} = (0.665,\ 0.245,\ 0.090)$$

Bẫy cài đặt

Bẫy	Hậu quả	Cách tránh
$\log 0$ khi p làm tròn thành 0 hoặc 1	Loss `inf` hoặc `nan`	Kẹp p vào $[\varepsilon, 1-\varepsilon]$
`np.exp` trên logit lớn	Softmax `nan`	Trừ max, log-sum-exp
Nộp `predict` cho metric log loss	$0.371 \to 5.407$	Nộp `predict_proba`
Đặc trưng lệch thang đo	Hội tụ chậm, phạt không đều	`StandardScaler` trong pipeline

Khi đi thi

- **Log loss**: nộp xác suất, kẹp trước khi nộp. Logistic regression cho xác suất khá đáng tin.
- **F1, lớp hiếm**: chọn ngưỡng trên validation; thử `class_weight="balanced"`.
- **Văn bản**: TF-IDF + logistic regression là baseline rất mạnh (bài 6.1).
- **Dữ liệu bảng**: `make_pipeline(StandardScaler(), LogisticRegression(C=...))`, dò `C` trên thang log.
- **Stacking**: logistic regression là tầng cuối an toàn (bài 8.4).

Tóm tắt

- Tuyến tính cho log-odds: $p = \sigma(\mathbf{w}^\top \mathbf{x} + b)$; mỗi đơn vị của x_j nhân odds e^{w_j}.
- Cross-entropy từ maximum likelihood; lồi; gradient $\frac{1}{n} X^\top (\mathbf{p} - \mathbf{y})$.
- Ranh giới vẫn thẳng; ngưỡng chọn theo cái giá của từng loại sai.
- Softmax cho K lớp, gradient $\mathbf{p} - \mathbf{e}_y$; trừ max, kẹp xác suất.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook của bài: thêm đặc trưng x_1^2, x_2^2, $x_1 x_2$ cho dữ liệu spam, xem ranh giới và log loss thay đổi thế nào.